Objektni dizajn
eSportskiCentar

Milan Aleksić
Saša Savić
Aldin Tursić
Igor Grubiša
Milan Ignjatić

Sadržaj

1. Uvod 2
2. Podsistem za rad sa administratorima 4
3. Podsistem za rad sa upravnicima 5
4. Podsistem za rad sa dežurnim radnicima 6
5. Podsistem za rad sa klijentima 7
6. Podsistem za upravljanje podacima 9

1. Uvod

U sistemu eSportskiCentar postoji ukupno 5 podsistema:

1) Podsistem za administratora

2) Podsistem za upravnika

3) Podsistem za dežurnog radnika

4) Podsistem za klijenta i

5) Podsistem za upravljanje podacima

U nastavku dokumenta, biće predstavljeni dijagrami klasa svih ovih podsistema, kao i dijagram klasa za upravljanje podacima. Prije samih dijagrama, biće detaljno objašnjena upotreba notacije za dijagrame klasa, što uključuje stereotipe i tipove podataka. Napominjemo da na dijagramima nisu nužno prikazani apsolutno svi modalni prozori. Razlog takve odluke je čitljiviji prikaz samog dijagrama, jer modalni prozori se odnose na post i put zahtjeve pa je iz relacionog modela koji se nalazi u dokumentu „Dizajn Sistema"dovoljno intuitivno i jasno koja input polja su u pitanju. Komponente koje sačinjavaju pogled su navedene kao određeni tipovi koji na dovoljno razumljiv način daju sam opis vizuelnog izgleda. Svaka akcija (dodavanje, izmjena i brisanje) daju povratnu notifikaciju o uspješnosti izvršene akcije, što takođe na dijagramima nije prikazano. Izostavljen je i input čija je uloga pretraga. On je dostupan na svakoj tabelarnoj komponenti i pretraživanje je napravljeno pogodno u odnosu na tipove podataka sadržane u tabeli.

Kod kritičnih operacija, kao što je brisanje nekog entiteta, ili blokiranje nekog naloga postoji potvrda za te opcije, ali su u dijagramima podsistema izostavljene zbog kompleksnosti samih dijagrama. Nazivi atributa unutar entiteta, modala i drugih oblika stereotipa su isti kao i u programskom kodu, dok su nazivi “onClick” metoda promijenjeni u cilju razumljivosti i preglednosti samih dijagrama. Svi podsistemi posjeduju SideBar sa svim funkcionalnostima namijenjenim za određeni podsistem, pa su njegove funkcionalnosti predstavljene i na dijagramima. Stereotipi korišćeni na dijagramima klasa:

<<**use**>> označava da servis ili stranica rade sa nekim perzistentnim tipom (entitet čuvan u bazi podataka)

<<**entity**>> domenski objekat koji se čuva u relacionoj bazi podataka

<<**link**>> veza između stranica i modala (označava da sa jedne stranice možemo doći na drugu)

<<page>> pogled (html stranica)

<<modal>> predstavlja dijalog prozor sa formom i dodatnim opcijama (otvoren je na linku tekućeg page-a ali je zasebna komponenta)

<<dialog>> dijalog prozor koji sadrži informacije

<<record>> "lažni" entiteti koji se ne čuvaju u bazi, ali se koriste od strane drugih entiteta i servisa

<<GET>> označava funkciju čiji je rezultat GET request

<<POST>> označava funkciju čiji je rezultat POST request

<<PUT>> označava funkciju čiji je rezultat PUT request

<<DELETE>> označava funkciju čiji je rezultat DELETE request

Tipovi podataka korišćeni na dijagramima klasa:

button - HTML klikabilno polje, odnosno dugme koje obavlja neku funkciju (korišćeno je za apstrakciju komponenti čija je glavna funkcija redirekcija nakon klika)

h1 - HTML veliki naslov, odnosi se na sadržaj pogleda

table - HTML tabela koja reprezentuje entitet iz baze podataka

div - HTML skup koda koji čini cjelinu

input - HTML označava polje za unos

timeline - prikaz događaja hronološki za jedan dan u smislenoj cjelini

card - predstavlja karticu na kojoj se nalaze sve bitne informacije u zavisnosti od entitetskog tipa

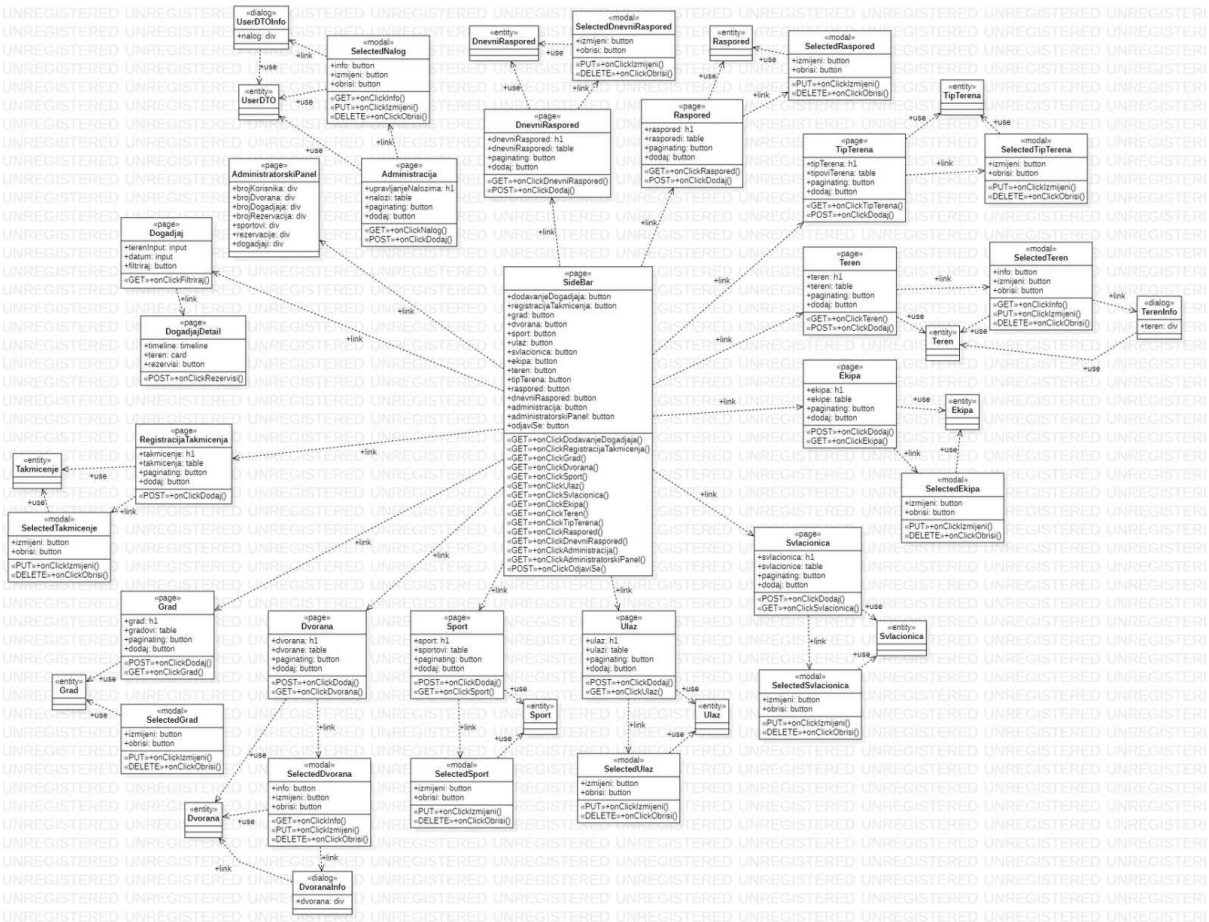
h3 - HTML srednji naslov, koristi se kao podnaslov

dropDown - komponenta padajućeg menija koji čine kompletni objekti, a ne samo stringovi

cards - reprezantacija više komponenti tipa card (niz card-ova)

2. Podsistem za rad sa administratorima

Podsistem za rad sa administratorima obezbjeđuje administratoru sistema funkcionalosti kao što su upravljanje nalozima različitog tipa (Administrator, Dežurni radnik i Upravnik – *Administracija* page), bilo to njihovo dodavanje, izmjena ili brisanje. Takođe, administrator sistema ima opciju i da blokira nekog od korisnika, ukoliko to nije neki drugi administrator. Pored upravljanja nalozima, administrator ima najveću kontrolu nad sistemom, jer posjeduje mogućnost čitanja, izmjene, dodavanja, brisanja rasporeda, dvorana, ekipa, terena, svlačionica, sportova itd. S obzirom da ima direktan pristup navedenim informacijama, ovaj podsistem pristupa podacima klasa *Raspored*, *Dvorana*, *Ekipa*, *Teren*, *Svlacionica*, *Sport* itd.



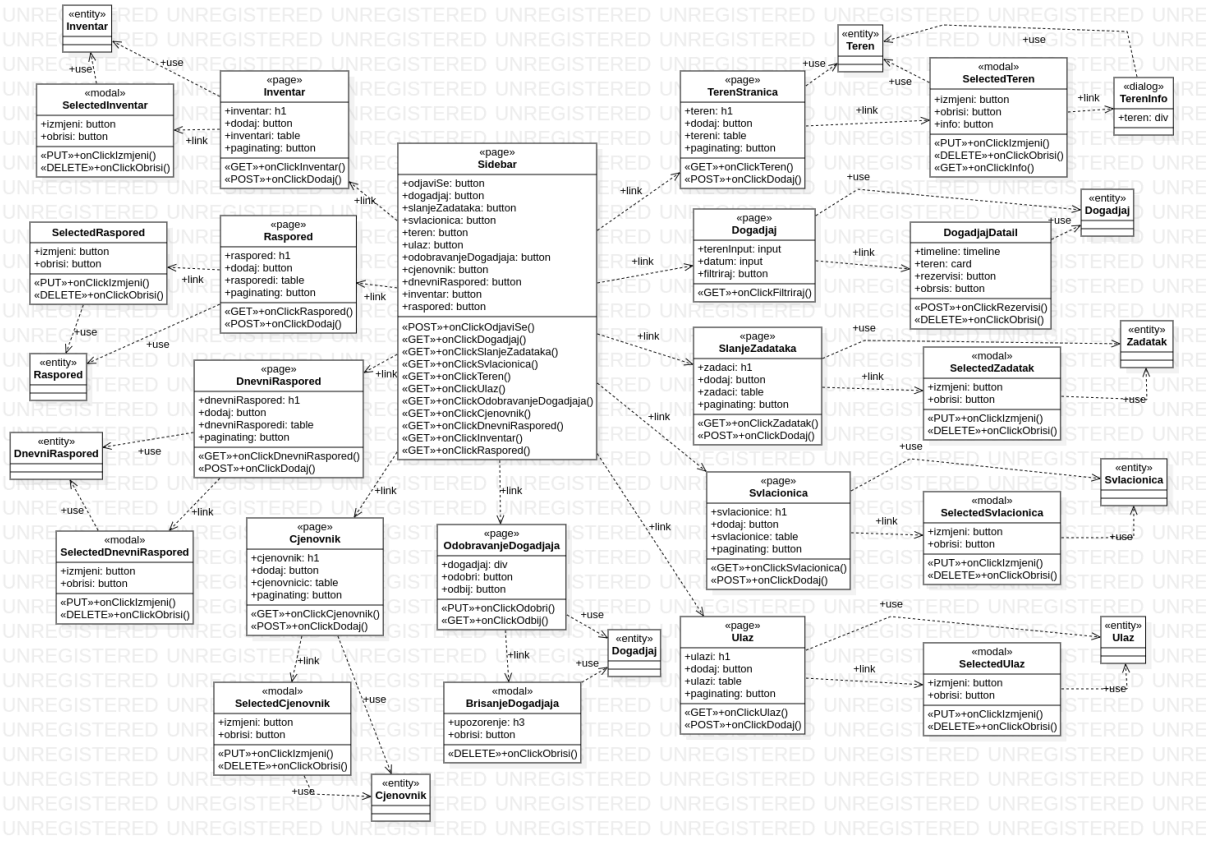
<< Dijagram klasa – podsistem za administratore >>

Na slici iznad prikazan je dijagram klasa za navedeni podsistem. Bitno je napomenuti da na svim stranicama (<<page>>) koje sadrže tabele sa podacima postoji mogućnost odabira željenog podatka te akcije koja će biti izvršena nad tim podatkom. Na primjer, u slučaju stranice "Dvorana", administrator ima mogućnost da prikaže informacije o nekoj dvorani, ali i da izmijeni podatke ili obriše dvoranu sa sistema. U programskom kodu ovo je predstavljeno dugmadima

za svako polje u tabeli, ali u slučaju dijagrama odabrana dvorana je predstavljena <<modal>> stereotipom radi lakše razumljivosti dijagrama i podsistema.

3. Podsistem za rad sa upravnicima

Podsistem za rad sa upravnikom je nešto više ograničen u odnosu na podsistem za administratore. Za razliku od administratora, upravnici nemaju kontrolu nad nalozima ali imaju pristup ostalim funkcionalnostima u sistemu. Osnovna uloga upravnika u sistemu je rad sa događajima koji uključuje njihov pregled (*Dogadjaj* i *DogadjajDetail*), kao i njihovo odobravanje (*OdobravanjeDogadjaja*), te slanje zadataka dežurnim radnicima (*SlanjeZadataka*). Aplikacija automatski provjerava da li je događaj moguće izvršiti pa upravnik ima opciju da dopusti ili odbije događaj. Upravnik takođe ima kontrolu nad cjenovnikom kog može mijenjati kada god poželi. Pored pristupa klasama kao u podsistemu za administratora, upravnik ima pristup i klasama *Inventar*, *Cjenovnik* i *Zadatak*, samo ako su u relaciji sa njegovom dvoranom.

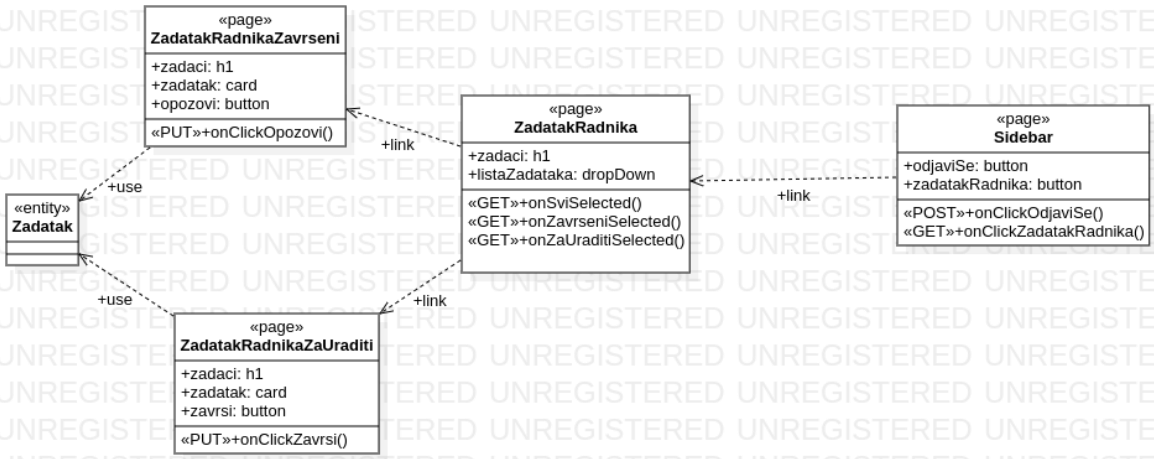


<< Dijagram klasa – podsistem za upravnika >>

Na osnovu slike iznad, na osnovu SideBar-a i njegovih funkcionalnosti možemo da primijetimo analogiju sa podsistemom za rad sa administratorima. Dakle, logika je slična.

4. Podsistem za rad sa dežurnim radnicima

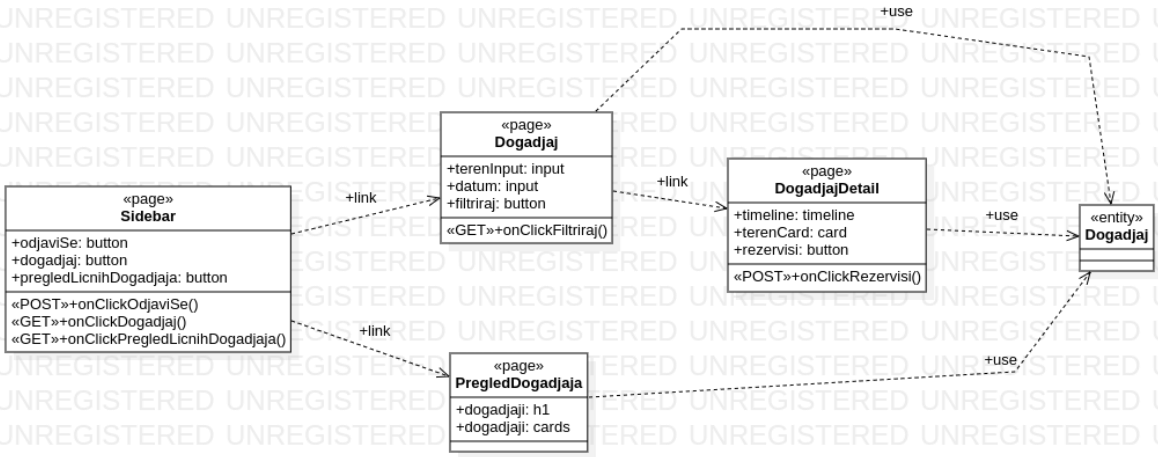
Podsistem za rad sa dežurnim radnikom je znatno jednostavniji u odnosu na prethodna dva sistema. Dežurni radnik na sistemu ima samo jedan zadatak, a to je da izvršava zadatke koje mu je zadao upravnik. Radnik ima pristup listi zadataka (*ZadatakRadnika*) koja sadrži zadatke koje radnik treba da izvrši, te zadatke koje je već završio. Radnik može da prilagodi listu da prikaže samo jedno od navedenog. Pristupa se podacima klase *Zadatak*.



<< Dijagram klasa – podsistem za rad sa dežurnim radnikom >>

5. Podsistem za rad sa klijentima

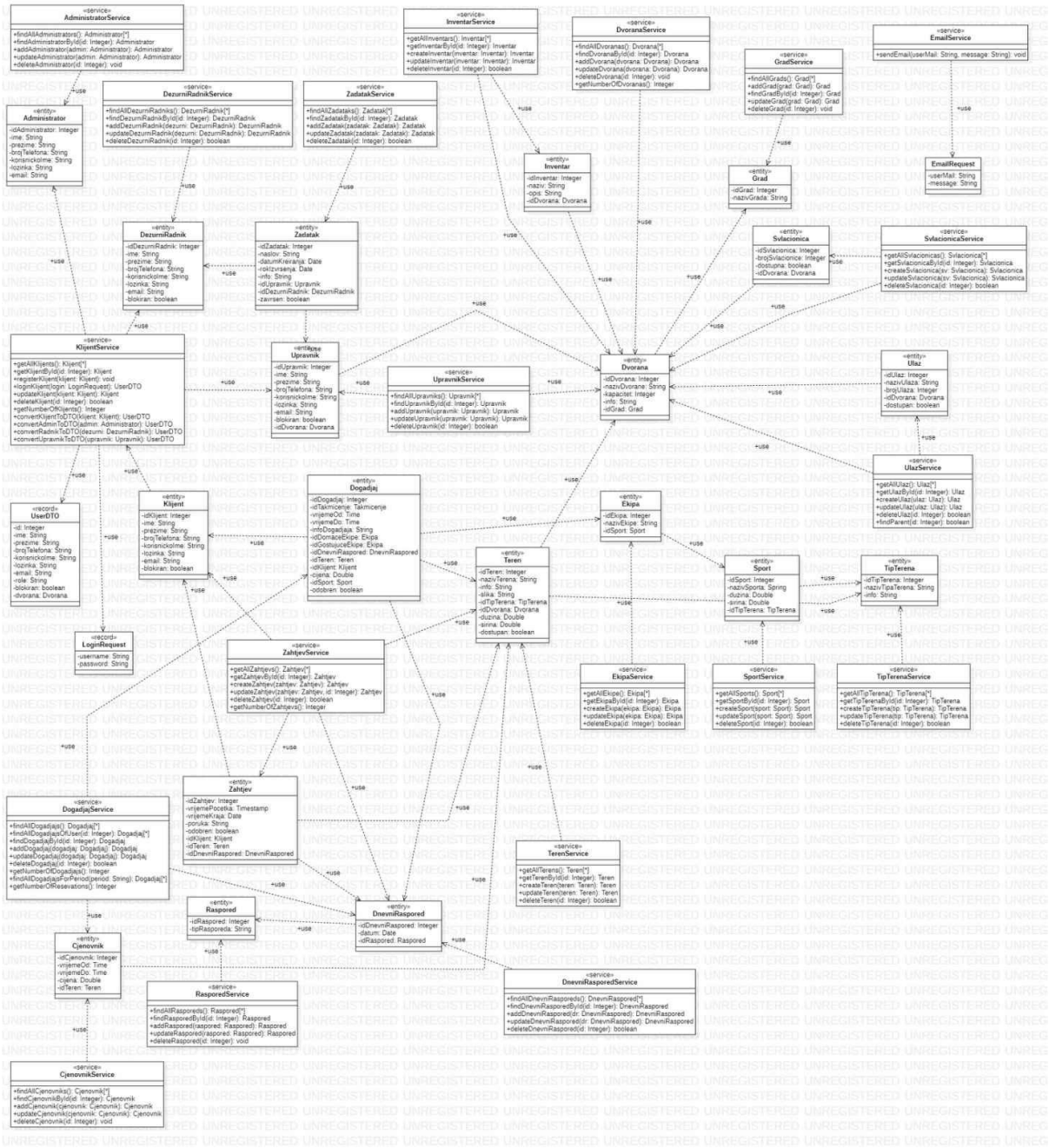
Klijent za razliku od ostalih tipova korisnika ima ograničen pristup. Svaki klijent ima opciju da prikaže detalje o događajima (*Dogadjaj* i *DogadjajDetail*) te da pregleda svoje događaje koji su mu odobreni od strane nekog od upravnika na sistemu (*PregledDogadjaja*). Dakle, pristupa se podacima klase *Dogadjaj*.



<< Dijagram klasa – podsistem za rad sa klijentima >>

Bitno je napomenuti da se klijenti samostalno registruju na sistem, dok je za ostale tipove naloga (Administrator, Dežurni radnik, Upravnik) zadužen neki od administratora.

6. Podsistem za upravljanje podacima



<< Dijagram klasa – podsistem za upravljanje podacima >>

Na sljedećem dijagramu prikazan je skup entiteta i servisa koji su međusobno povezani u odnosu na njihovu primjenu. Ovim dijagramom se prikazuje upravljanje nad podacima u bazi podataka na osnovu odgovarajućih servisa. Dakle, na dijagramu kao prvo postoje klase koje reprezentuju entitete u bazi podataka. Ove klase su predstavljene stereotipom <<entity>>. Da bi vršili kontrolu/manipulaciju nad entitetima, svaki entitet kontroliše odgovarajući servis (<<service>>) uz pomoć veze <<use>>. Ovo ne znači da servisi ne mogu da koriste druge entitete pored entiteta za kog su namijenjeni. Takođe, entiteti mogu

međusobno da posjeduju vezu <<use>> ukoliko postoji strani ključ u jednom od entiteta. Servisi imaju pristup drugim entitetima ukoliko u backend-u imaju pristup repozitorijumima kreiranim za navedene entitete.

Pored standardnih servisa kreiranih isključivo za kontrolu nad podacima u bazi, postoje i servisi druge namjene, kao što je npr. *EmailService* koji je namijenjen za slanje mejlova između radnika na sistemu.

Svaki od servisa u podsistemu sadrži osnovne metode za manipulaciju nad podacima kao što su *getAll( )*, *getById( )*, *add( )*, *update( )* i *delete( )*. Na osnovu ovih operacija se pristupa repozitorijumu koji na osnovu njih vrši određene radnje nad podacima u bazi.

Pored servisa i entiteta postoji još jedan stereotip, a to su rekordi (<<record>>). Njih možemo posmatrati kao “lažne” entitete koji se ne čuvaju u bazi, nego služe isključivo za formiranje specifičnih objekata u runtime-u kako bi mogli biti primijenjeni od strane drugih entiteta i servisa. Primjer ovoga su rekordi *LoginRequest* koji predstavlja kredencijale prilikom prijave na sistem, kao i *UserDTO* koji služi kao pomoć prilikom prijave korisnika na sistem. *UserDTO* na osnovu dobijenih podataka određuje koji tip naloga se prijavio na system (atribut *Role*), da li to bio administrator, dežurni radnik, upravnik ili klijent.